TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS • BACK-END

Persistindo dados: JSON e PostgreSQL

O CRUD da aula passada, agora com dados que sobrevivem ao reinício do servidor

```
const { rows } = await pool.query(
  'SELECT * FROM contatos'
);
```

PROFESSOR

Tarik Ponciano

BACK-END COM NODE.JS

Roteiro

01 **Por que persistir**
Memória, arquivo ou banco: onde os dados moram

02 **Arquivo JSON**
Módulo fs, async/await e o CRUD gravando num arquivo

03 **PostgreSQL**
Biblioteca pg, consultas com parâmetros e o CRUD em SQL

04 **Fechamento**
As três versões lado a lado, desafio e erros comuns

NESTE MATERIAL

Exemplo explicado
O professor monta um servidor novo: a agenda de contatos.

Prática
Você monta outro, do zero: a loja de jogos.

Código JavaScript
Com import, pronto para o server.js.

ONDE ESTAMOS

Da memória para o banco

AULA PASSADA • MEMÓRIA

Lista no servidor

```
let alunos = [
  { id: 1, nome: 'Ana' }
];
```

Rápido de fazer, mas **some** quando o servidor reinicia.

HOJE • PARTE 1

Arquivo JSON

```
await writeFile(
  'contatos.json', texto
);
```

Os dados ficam num **arquivo** dentro do projeto.

HOJE • PARTE 2

PostgreSQL

```
await pool.query(
  'SELECT * FROM contatos'
);
```

Os dados ficam no **banco** que você já conhece do pgAdmin.

As rotas, os métodos e o JSON de entrada e saída **continuam os mesmos**. Muda só onde os dados moram.

SEÇÃO 01

Por que persistir

01

Memória, arquivo ou banco: onde os dados moram — e por que isso importa.

01 • Persistência
02 • Arquivo JSON
03 • PostgreSQL
04 • Fechamento

01 • PERSISTÊNCIA

Reiniciou, perdeu

UMA TARDE NA API DE ALUNOS

1. `POST /alunos`
   {"nome": "Carla", "turma": "TDS-01"}
   201 • id 3

2. `Ctrl + C · node server.js`
   o servidor reinicia (ou o --watch reinicia ao salvar)

3. `GET /alunos/3`
   {"erro": "Aluno não encontrado"}
   404

A lista **let alunos = [...]** mora na memória do processo. Quando o processo termina, a memória é liberada — e a Carla vai junto.

> **Persistir**
> Guardar os dados **fora do processo**, num lugar que continua lá depois do reinício: um arquivo ou um banco de dados.

01 • PERSISTÊNCIA

Três lugares para os dados

	Memória	Arquivo JSON	PostgreSQL
Onde ficam	Na memória do processo	Num arquivo do projeto	No servidor de banco
Reiniciou o servidor	Perde tudo	Continua lá	Continua lá
Como buscar	find e filter	Lê o arquivo inteiro e filtra	SQL: WHERE, ORDER BY
Bom para	Testes e protótipos	Projetos pequenos	Sistemas com muitos usuários

Hoje você vai fazer o **mesmo CRUD** duas vezes: primeiro gravando num arquivo, depois no PostgreSQL.

OS CASOS DE HOJE

Dois servidores novos, do zero

EXEMPLO DO PROFESSOR

Agenda de contatos

`api-contatos · /contatos`

`nome` `telefone`

Resolvida nos slides de exemplo, passo a passo: primeiro com arquivo, depois com banco.

SUA PRÁTICA

Loja de jogos

`api-jogos · /jogos`

`titulo` `plataforma` `preco`

Você faz o mesmo passo em cada prática, no seu próprio servidor.

PROJETO NOVO

```
mkdir api-jogos
npm init -y
npm pkg set type=module
npm install express
```

SEÇÃO 02

Arquivo JSON

02

Gravar a lista num arquivo do projeto, com o módulo fs do próprio Node.

02 · ARQUIVO JSON

O módulo fs: o Node mexe em arquivos

```
import { readFile, writeFile } from 'node:fs/promises';

// lê o arquivo inteiro como texto
const texto = await readFile('./contatos.json', 'utf-8');

// grava o texto no arquivo (substitui o que tinha)
await writeFile('./contatos.json', texto);
```

node:fs/promises

Já vem com o Node: nada de npm install. O **node:** indica um módulo do próprio Node.

readFile

Devolve o conteúdo do arquivo. Com **'utf-8'**, ele vem como texto.

writeFile

Troca **todo** o conteúdo do arquivo pelo texto novo.

```
api-contatos/
├── contatos.json      novo
├── node_modules/
├── package.json
└── server.js
```

02 · ARQUIVO JSON

Texto e objeto: JSON.parse e JSON.stringify

LER

TEXTO · NO ARQUIVO

```
[{"id":1,"nome":"Ana"}]
```

JSON.parse(texto) →

OBJETO · NO JAVASCRIPT

```
[{ id: 1, nome: 'Ana' }]
```

GRAVAR

OBJETO · NO JAVASCRIPT

```
[{ id: 1, nome: 'Ana' }]
```

JSON.stringify(contatos) →

TEXTO · NO ARQUIVO

```
[{"id":1,"nome":"Ana"}]
```

Por que converter?

O arquivo só guarda texto. Para usar find, push e filter, o texto precisa virar objeto — e voltar a ser texto para gravar.

E o null, 2?

O **2** organiza o arquivo com 2 espaços de recuo. Fica legível para você abrir e conferir.

02 · ARQUIVO JSON

Por que async e await?

- Ler e gravar arquivo **leva tempo**. O Node não fica parado esperando.
- **await** espera o resultado antes de seguir para a próxima linha.
- Função que usa await precisa ser **async**.

Você já usou: o Inquirer também espera a resposta do teclado.

```
const nome = await input({ ... });
```

```
app.get('/contatos', async (req, res) ⇒ {
                     1
  const contatos = await lerContatos();
                         2
  res.json(contatos);
});
```

1 • async
Avisa que a função vai esperar alguma coisa.

2 • await
Espera a leitura do arquivo terminar.

⚠ Esqueceu o await? A rota recebe uma **Promise**, não a lista — e responde {}.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Duas funções: ler e salvar

SERVER.JS · ANTES DAS ROTAS

```
import { readFile, writeFile } from 'node:fs/promises';

const ARQUIVO = './contatos.json';

async function lerContatos() {
  const texto = await readFile(ARQUIVO, 'utf-8');
  return JSON.parse(texto);
}

async function salvarContatos(contatos) {
  const texto = JSON.stringify(contatos, null, 2);
  await writeFile(ARQUIVO, texto);
}
```

contatos.json

```
[
  {
    "id": 1,
    "nome": "Ana",
    "telefone": "85999990001"
  }
]
```

Assim fica o arquivo gravado com stringify(contatos, null, 2).

⚠ Comece o arquivo com **[]**. Vazio, o JSON.parse dá erro.

02 • ARQUIVO JSON • NA NUVEM

Caminho certo e arquivo garantido

SERVER.JS • NO LUGAR DO const ARQUIVO

```
import { readFile, writeFile } from 'node:fs/promises';
import { existsSync } from 'node:fs';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

// a pasta do server.js, em qualquer computador
const PASTA = path.dirname(fileURLToPath(import.meta.url));
const ARQUIVO = path.join(PASTA, 'contatos.json');

// primeiro deploy? cria o arquivo com uma lista vazia
if (!existsSync(ARQUIVO)) {
  await writeFile(ARQUIVO, '[]');
}
```

O problema

'./contatos.json' é procurado na pasta de onde o node foi iniciado, não na do server.js. Na nuvem, ele pode iniciar de outra pasta: ENOENT.

`import.meta.url`
O endereço do próprio server.js.

`fileURLToPath + dirname`
Viram a pasta onde ele está.

`path.join`
Junta pasta e nome com a barra certa, no Windows ou no Linux.

`existsSync`
Confere se o arquivo existe antes do primeiro uso.

Node 20.11 ou mais novo: **import.meta.dirname** faz o papel da PASTA em uma linha.

⚠ Em muitas nuvens, como o plano gratuito do Render, o disco é apagado a cada deploy ou reinício. O arquivo serve para testes; para guardar de verdade, use um banco — a parte 2 da aula.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Ler: GET com o arquivo

```
app.get('/contatos', async (req, res) => {
  const contatos = await lerContatos();
  res.json(contatos);
});
```

```
app.get('/contatos/:id', async (req, res) => {
  const id = Number(req.params.id);
  const contatos = await lerContatos();
  const contato = contatos.find(a => a.id === id);
  if (!contato) {
    return res.status(404)
      .json({ erro: 'Contato não encontrado' });
  }
  res.json(contato);
});
```

O QUE MUDOU

async na função da rota
Porque ela vai esperar a leitura.

await lerContatos()
A lista vem do arquivo, a cada pedido.

O resto
Igual ao da aula passada.

TESTE

Abra o contatos.json, troque o nome da Ana à mão, salve e faça **GET /contatos/1** de novo.

PRÁTICA 1 • 25 MIN • INDIVIDUAL

A loja de jogos no arquivo

a. Crie o projeto novo **api-jogos**: "type": "module", Express e express.json().

b. Crie o **jogos.json** e as funções **lerJogos()** e **salvarJogos(jogos)**, com path.join.

c. **GET /jogos** e **GET /jogos/:id** lendo do arquivo, com 404.

d. Mude um preço direto no arquivo e faça **GET /jogos/1**.

COMEÇO DO jogos.json

```
[
  {
    "id": 1,
    "titulo": "Corrida Turbo",
    "plataforma": "Console",
    "preco": 149.9
  },
  ...
]
```

Depois: Reino Perdido (PC, 89.90) e Quiz Mestre (Celular, 19.90).

PENSE E RESPONDA Por que o GET viu a mudança no arquivo sem reiniciar o servidor?

02 · ARQUIVO JSON

Toda rota que altera: ler, mudar, salvar

1

Ler

`await lerContatos()`

Traz a lista do arquivo.

→

2

Mudar

`push · find · filter`

Igual à aula passada.

→

3

Salvar

`await salvarContatos()`

Grava a lista inteira de volta.

→

4

Responder

`res.status(...)`

Só depois de salvar.

Esqueceu de salvar?

A resposta sai certa, mas o arquivo não muda: no próximo GET, a alteração sumiu.

Por que salvar antes de responder?

Se a gravação falhar, o cliente não recebe um 201 de algo que não foi guardado.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Criar: POST no arquivo

```
app.post('/contatos', async (req, res) => {
  const { nome, telefone } = req.body;
  if (!nome || !telefone) {
    return res.status(400)
      .json({ erro: 'Informe nome e telefone' });
  }

  const contatos = await lerContatos();  // 1. ler
  const novo = { id: gerarId(contatos), nome, telefone };
  contatos.push(novo);  // 2. mudar
  await salvarContatos(contatos);  // 3. salvar

  res.status(201).json(novo);  // 4. responder
});
```

E O ID?

```
function gerarId(lista) {
  if (lista.length === 0) return 1;
  const ids = lista.map(a => a.id);
  return Math.max(...ids) + 1;
}
```

Por que não o proximoId?

Ele vivia na memória e voltava ao início a cada reinício. Agora o id novo é o maior id gravado + 1.

Teste

POST /contatos → 201. Abra o contatos.json: o contato novo está lá.

02 · ARQUIVO JSON · EXEMPLO EXPLICADO

Atualizar e remover no arquivo

PUT · ATUALIZAR

```
app.put('/contatos/:id', async (req, res) => {
  const id = Number(req.params.id);
  const contatos = await lerContatos();
  const contato = contatos.find(a => a.id === id);
  if (!contato) {
    return res.status(404)
      .json({ erro: 'Contato não encontrado' });
  }
  contato.nome = req.body.nome ?? contato.nome;
  contato.telefone =
    req.body.telefone ?? contato.telefone;
  await salvarContatos(contatos);
  res.json(contato);
});
```

DELETE · REMOVER

```
app.delete('/contatos/:id', async (req, res) => {
  const id = Number(req.params.id);
  const contatos = await lerContatos();
  const resto = contatos.filter(a => a.id !== id);
  if (resto.length === contatos.length) {
    return res.status(404)
      .json({ erro: 'Contato não encontrado' });
  }
  await salvarContatos(resto);
  res.status(204).send();
});
```

O find devolve o objeto da lista lida: mudar contato.nome muda a lista, e o salvar grava tudo.

O filter não tirou ninguém? Então o id não existia: 404.

PRÁTICA 2 • 25 MIN • INDIVIDUAL

A loja que não esquece

a. **POST /jogos**: 400 sem título ou preço; senão ler, criar com gerarId, salvar e 201.

b. **PUT /jogos/:id**: ler, trocar só o que veio no corpo, salvar.

c. **DELETE /jogos/:id**: ler, filtrar, salvar e responder 204 (ou 404).

Modelo: POST, PUT e DELETE de contatos com arquivo.

TESTES NO THUNDER CLIENT

`POST /jogos` — **201**

```
{ "titulo": "Fazenda Feliz",
  "plataforma": "Celular", "preco": 9.9 }
```

`PUT /jogos/1 · {"preco": 129.9}` — **200**

`DELETE /jogos/2` — **204**

Reinicie o servidor e faça GET /jogos:

a Fazenda Feliz continua lá; o Reino Perdido, não.

PENSE E RESPONDA Abra o jogos.json depois de cada teste. O que mudou no arquivo?

SEÇÃO 03

PostgreSQL

03

O mesmo CRUD, agora com o banco que você já conhece do pgAdmin.

03 · POSTGRESQL

Você já sabe SQL

Thunder Client
pede GET /contatos

→

Express
a rota roda

→

Biblioteca pg
leva o SQL ao banco

→

PostgreSQL
executa e devolve as linhas

Rota	Comando	Exemplo em SQL
GET /contatos	SELECT	`SELECT * FROM contatos;`
GET /contatos/:id	SELECT	`SELECT * FROM contatos WHERE id = 3;`
POST /contatos	INSERT	`INSERT INTO contatos (nome, telefone) VALUES ('Ana', '85999990001');`
PUT /contatos/:id	UPDATE	`UPDATE contatos SET telefone = '85988880002' WHERE id = 3;`
DELETE /contatos/:id	DELETE	`DELETE FROM contatos WHERE id = 3;`

03 · POSTGRESQL · EXEMPLO EXPLICADO

Preparando o banco no pgAdmin

QUERY TOOL · BANCO api_contatos

```
CREATE TABLE contatos (
  id       SERIAL PRIMARY KEY,
  nome     VARCHAR(100) NOT NULL,
  telefone VARCHAR(20)  NOT NULL
);

INSERT INTO contatos (nome, telefone) VALUES
  ('Ana', '85999990001'),
  ('Bruno', '85999990002');
```

1. No pgAdmin, crie o banco **api_contatos**.
2. Abra a Query Tool nesse banco e rode o código (ou crie a tabela pela interface).
3. Confira: **SELECT * FROM contatos;**

SERIAL: o banco gera o id

1, 2, 3… sozinho. Substitui o gerarId — por isso o INSERT não informa o id.

03 · POSTGRESQL · CONFIGURAÇÃO

Conectando com a biblioteca pg

DB.JS · ARQUIVO NOVO

```
import pg from 'pg';
const { Pool } = pg;

export const pool = new Pool({
  host: 'localhost',
  port: 5432,
  database: 'api_contatos',
  user: 'postgres',
  password: 'sua_senha'
});
```

⚠ Senha no código só em aula. Num projeto real, ela vai para variáveis de ambiente e nunca sobe para o GitHub.

1 · INSTALE

```
npm install pg
```

2 · CRIE O DB.JS

Com os dados de conexão do seu PostgreSQL.

3 · IMPORTE NO SERVER.JS

```
import { pool } from './db.js';
```

Pool

Um conjunto de conexões prontas: cada consulta pega uma, usa e devolve.

Usando a Aiven?

Copie host, porta, banco, usuário e senha do painel e acrescente **ssl: { rejectUnauthorized: false }**.

03 · POSTGRESQL · ANATOMIA

Anatomia de uma consulta

```
const { rows } = await pool.query(
        (1)                (2)
  'SELECT * FROM contatos WHERE id = $1',
                          (3)
  [id]
   (4)
);
```

1 • O resultado
rows: as linhas, cada uma como objeto. Também vem rowCount.

2 • A consulta
Manda o SQL ao banco e espera a resposta: por isso o await.

3 • O SQL
O mesmo da Query Tool, com $1 no lugar do valor.

4 • Os valores
Na ordem: o primeiro vai no $1, o segundo no $2…

O QUE VOLTA

```
rows = [ { id: 3, nome: 'Carla', telefone: '85999990003' } ]
```

03 • POSTGRESQL • SEGURANÇA

Valores sempre como parâmetros

EVITE • JUNTAR TEXTO

```
pool.query(
  'SELECT * FROM contatos WHERE id = '
  + req.params.id
);
```

Se alguém pedir **/contatos/1 OR 1=1**, o banco recebe outro comando e devolve todos os contatos. Isso é **injeção de SQL**.

PREFIRA • PARÂMETROS

```
pool.query(
  'SELECT * FROM contatos WHERE id = $1',
  [req.params.id]
);
```

O pg envia o valor **separado** do comando. O banco trata como dado, nunca como SQL.

Vários valores? **$1, $2, $3** no SQL, na mesma ordem do array: '... VALUES ($1, $2)', [nome, telefone]

Ler: GET com PostgreSQL

```
app.get('/contatos', async (req, res) => {
  const { rows } = await pool.query(
    'SELECT * FROM contatos ORDER BY id');
  res.json(rows);
});
```

```
app.get('/contatos/:id', async (req, res) => {
  const { rows } = await pool.query(
    'SELECT * FROM contatos WHERE id = $1',
    [req.params.id]);
  if (rows.length === 0) {
    return res.status(404)
      .json({ erro: 'Contato não encontrado' });
  }
  res.json(rows[0]);
});
```

O QUE MUDOU

lerContatos() → pool.query
O banco busca; nada de ler arquivo.

find → WHERE id = $1
O filtro sai do JavaScript e vai para o SQL.

rows[0]
A consulta sempre devolve uma lista, mesmo com uma linha só.

TESTE

Insira um contato pelo pgAdmin e faça **GET /contatos**: ele aparece na API.

PRÁTICA 3 • 25 MIN • INDIVIDUAL

A loja de jogos no PostgreSQL

a. No pgAdmin: banco **api_jogos**, a tabela ao lado e os três jogos com INSERT.

b. Na api-jogos: **npm install pg** e um **db.js** apontando para o api_jogos.

c. Reescreva **GET /jogos** e **GET /jogos/:id** com pool.query.

d. Teste: /jogos/2 dá **200**; /jogos/99 dá **404**.

A TABELA

```
CREATE TABLE jogos (
  id          SERIAL PRIMARY KEY,
  titulo      VARCHAR(100) NOT NULL,
  plataforma  VARCHAR(30),
  preco       NUMERIC(6,2) NOT NULL
);
```

Jogos: Corrida Turbo (Console, 149.90), Reino Perdido (PC, 89.90), Quiz Mestre (Celular, 19.90).

PENSE E RESPONDA O preço voltou como "149.90", entre aspas. Por que o pg devolve NUMERIC como texto?

Criar: INSERT com RETURNING

```
app.post('/contatos', async (req, res) => {
  const { nome, telefone } = req.body;
  if (!nome || !telefone) {
    return res.status(400)
      .json({ erro: 'Informe nome e telefone' });
  }

  const { rows } = await pool.query(
    `INSERT INTO contatos (nome, telefone)
     VALUES ($1, $2) RETURNING *`,
    [nome, telefone]
  );
  res.status(201).json(rows[0]);
});
```

RETURNING *

O banco devolve a linha que acabou de criar, com o id que o SERIAL gerou.

Crase: SQL em várias linhas

O texto entre `` pode quebrar linha. Fica mais fácil de ler.

Sem ler, mudar e salvar

O INSERT já grava. O banco faz as três etapas sozinho.

Teste

POST /contatos → 201 com o id novo. Confira no pgAdmin.

03 · POSTGRESQL · EXEMPLO EXPLICADO

Atualizar e remover com SQL

PUT · ATUALIZAR

```
app.put('/contatos/:id', async (req, res) => {
  const { nome, telefone } = req.body;
  const { rows } = await pool.query(
    `UPDATE contatos
     SET nome = COALESCE($1, nome),
         telefone = COALESCE($2, telefone)
     WHERE id = $3 RETURNING *`,
    [nome, telefone, req.params.id]);
  if (rows.length === 0) {
    return res.status(404)
      .json({ erro: 'Contato não encontrado' });
  }
  res.json(rows[0]);
});
```

DELETE · REMOVER

```
app.delete('/contatos/:id', async (req, res) => {
  const { rowCount } = await pool.query(
    'DELETE FROM contatos WHERE id = $1',
    [req.params.id]);
  if (rowCount === 0) {
    return res.status(404)
      .json({ erro: 'Contato não encontrado' });
  }
  res.status(204).send();
});
```

COALESCE($1, nome): usa o valor novo; se ele não veio, mantém o atual. É o ?? do SQL.

rowCount: quantas linhas o DELETE apagou. Zero? O id não existia: 404.

03 · POSTGRESQL · EXEMPLO EXPLICADO

Quando o banco falha: try e catch

```
app.get('/contatos', async (req, res) ⇒ {
  try {
    const { rows } = await pool.query(
      'SELECT * FROM contatos ORDER BY id');
    res.json(rows);
  } catch (erro) {
    console.error(erro.message);
    res.status(500).json({ erro: 'Erro no banco de dados' });
  }
});
```

try

Tenta rodar o bloco normalmente.

catch

Se algo der erro lá dentro, cai aqui. O motivo vai para o terminal; o cliente recebe um 500 em JSON.

Quando acontece

Banco desligado, senha errada, nome de tabela errado.

Teste: troque contatos por contato no SQL e faça GET /contatos. O terminal mostra relation "contato" does not exist.

PRÁTICA 4 · 25 MIN · INDIVIDUAL

CRUD completo no banco

a. **POST /jogos** com INSERT … RETURNING *.

b. **PUT /jogos/:id** com UPDATE e COALESCE.

c. **DELETE /jogos/:id** usando o rowCount.

d. **try/catch** nas rotas: erro no banco vira 500.

Modelo: o CRUD de contatos com PostgreSQL.

TESTES NO THUNDER CLIENT

`POST /jogos` — **201**

```
{ "titulo": "Fazenda Feliz",
  "plataforma": "Celular", "preco": 9.9 }
```

`PUT /jogos/1 · {"preco": 129.9}` — **200**

`DELETE /jogos/2` — **204**

`GET /jogos/2` — **404**

Depois, no pgAdmin: **SELECT * FROM jogos;**

PENSE E RESPONDA Mude um preço pelo pgAdmin e faça GET pelo Thunder. Quem guarda a verdade dos dados agora?

SEÇÃO 04

Fechamento

04

As três versões lado a lado, o desafio em dupla e os erros mais comuns.

04 · FECHAMENTO

Mesma rota, três jeitos

Dentro de **GET /contatos/:id**, só muda a parte que busca o contato:

MEMÓRIA

```
const contato = contatos
  .find(a ⇒ a.id ═══ id);
```

Some ao reiniciar o servidor.

ARQUIVO JSON

```
const contatos =
  await lerContatos();
const contato = contatos
  .find(a ⇒ a.id ═══ id);
```

Sobrevive. Bom para projetos pequenos.

POSTGRESQL

```
const { rows } =
  await pool.query(
  `SELECT * FROM contatos
   WHERE id = $1`, [id]);
const contato = rows[0];
```

Sobrevive. Feito para muitos usuários.

O resto da rota — o 404 e o res.json(contato) — é **igual nas três**. Muda só onde os dados moram.

PRÁTICA 5 • DESAFIO • 50 MIN • DUPLAS

A API do desafio, agora no banco

Peguem a API que a dupla fez no desafio da aula passada e troquem a lista em memória pelo **PostgreSQL**.

Cinema
`filmes`
titulo, genero, duracao

Biblioteca
`livros`
titulo, autor, disponivel

Academia
`clientes`
nome, plano, ativo

Pet shop
`pets`
nome, especie, tutor

Oficina
`ordens`
placa, servico, status

Tarefas
`tarefas`
descricao, concluida

ENTREGAS

1. Tabela no pgAdmin, com id SERIAL e os campos do cenário.
2. As cinco rotas com pool.query e $1, $2…
3. Status 201, 204, 400, 404 — e 500 com try/catch.
+ **Desafio:** filtro com WHERE, como /filmes?genero=comedia.

PENSE E RESPONDA Que tipo de coluna vocês escolheram para cada campo? Por quê?

04 · FECHAMENTO

Erros comuns e como resolver

Sintoma	Causa provável	Solução
ENOENT: no such file or directory	Arquivo não existe, ou o node rodou de outra pasta	Monte o caminho com path.join e existsSync
Unexpected end of JSON input	O arquivo .json está vazio	Escreva [] dentro dele
A rota responde {}	Faltou o await	await na chamada e async na rota
A alteração some no próximo GET	Mudou a lista, mas não salvou	Chame o salvar antes de responder
Servidor reinicia a cada POST	O nodemon vigia arquivos .json	Use node --watch
password authentication failed	Usuário ou senha errados no db.js	Confira os dados de conexão
relation "contatos" does not exist	Tabela não criada, ou banco errado no db.js	Confira o database e a tabela
invalid input syntax for type integer	O id da rota não é número: /contatos/dois	Valide o id e responda 400
Preço volta como "6.50"	O pg devolve NUMERIC como texto	Converta com Number() se precisar

04 • FECHAMENTO

Confira antes de chamar o professor

ARQUIVO JSON

- [ ] Caminho com path.join; arquivo criado se faltar.
- [ ] Ler e salvar com await, em rotas async.
- [ ] Toda rota que altera: ler, mudar, salvar.

POSTGRESQL

- [ ] Tabela com id SERIAL criada no banco certo.
- [ ] db.js conectado: o servidor sobe sem erro.
- [ ] Todo valor como $1, $2 — nada de juntar texto.

TESTES

- [ ] Dados continuam depois de reiniciar o servidor.
- [ ] Status 200, 201, 204, 400, 404 e 500 conferidos.
- [ ] Mudanças aparecem no pgAdmin.

BACK-END · CONSULTA RÁPIDA

Resumo

Persistir
fora do processo
arquivo ou banco
sobrevive ao reinício

fs
node:fs/promises
readFile(c, 'utf-8')
writeFile(c, texto)

JSON
parse: texto → objeto
stringify: o inverso
(dados, null, 2)

async / await
await espera
quem espera é async
sem await: {}

Arquivo
ler · mudar · salvar
gerarId: maior + 1
comece com []

pg
npm install pg
new Pool({ ... })
export const pool

Consulta
pool.query(sql, [v])
{ rows, rowCount }
um item: rows[0]

Parâmetros
$1, $2 na ordem
nunca juntar texto
evita injeção

Criar
id SERIAL
INSERT sem o id
RETURNING *

Atualizar
COALESCE($1, nome)
o ?? do SQL
sem linhas: 404

Remover
DELETE … $1
rowCount 0: 404
senão: 204

Erros
try / catch
500 em JSON
console.error

BACK-END • AULA CONCLUÍDA

Seus dados sobrevivem!

Termine o CRUD da loja de jogos no banco e o desafio em dupla.

PARA ESTUDAR

nodejs.org/api/fs.html — documentação do módulo fs

node-postgres.com — documentação da biblioteca pg

postgresql.org/docs — documentação do PostgreSQL

expressjs.com/pt-br — documentação do Express